sarion
WORK SMARTER. ACHIEVE MORE.

SARION CLAUDE CODE MASTERY KIT · VERSION 1.0.0

SARION Claude Code Mastery Kit — Free Sample

A preview of the prompt library, debugging playbooks, and architecture guides inside the complete kit

PROFESSIONAL EDITION

Work Smarter. Achieve More.

About This Book

DIFFICULTY **All Levels**	READING TIME **~24 min**
PAGES **12+**	CATEGORY **Sample Edition**
VERSION **1.0.0**	UPDATED **July 2026**
PREREQUISITES **None**	AUDIENCE **Prospective readers**

About SARION

Mission. We build premium, developer-first tools that help engineers get radically more out of AI. Every SARION product is opinionated, battle-tested, and built to be used on day one — not read once and shelved.

Vision. A world where every developer can build production software at senior-engineer speed and quality, with AI as a true engineering partner rather than a novelty autocomplete.

This book is one part of the SARION Claude Code Mastery Kit — a 300+ page library of prompts, playbooks, templates, and handbooks for developers who build with Claude Code.

SARION · *Work Smarter. Achieve More.*

SARION Claude Code Mastery Kit — Free Sample

Part of the **SARION Claude Code Mastery Kit**.

Version	**1.0.0**
Released	**July 2026**
Publisher	**SARION**
License	**Commercial, single-seat (see LICENSE.md)**
Website	**trysarion.com**
Support	**support@trysarion.com**
Documentation	**Included — see START-HERE.md**
GitHub	**github.com/techsarion**

All code samples are provided for educational and production reference purposes. Always review and test AI-generated code before deploying to production systems.

Table of Contents

🏛️ The Architecture Playbook

CHAPTER 01

The SARION Prompt Library — 250+ Claude Code Prompts

SARION

Work Smarter. Achieve More.

🧠 The SARION Prompt Library — 250+ Claude Code Prompts

> The heart of the SARION Claude Code Mastery Kit. Over **250 production-quality, copy-paste prompts** across 12 categories — engineered to save you significant time on every task.

📖 How To Use This Library

1. **Find your category** in the table below.
2. **Open the file** and scan the prompt titles.
3. **Copy the prompt** from the `## Claude Code Prompt` block.
4. **Replace every** `{{PLACEHOLDER}}` with your real context.
5. **Paste into Claude Code** and iterate.

Every prompt follows the same premium structure:

FIELD	WHAT IT GIVES YOU
Purpose	What the prompt does
When to Use	The exact scenario it fits
Claude Code Prompt	The full copy-paste text
Expected Output	What Claude should produce
Pro Tips	Expert tweaks for better results
Example	A realistic, worked example

📚 Categories

#	CATEGORY	FOCUS	FILE
01	**Project Setup**	Scaffolding, config, initialization	01-Project-Setup.md
02	**Full Stack Development**	CRMs, dashboards, marketplaces, billing	02-Full-Stack-Development.md
03	**Frontend Development**	React, Next.js, Tailwind, UI, charts	03-Frontend-Development.md
04	**Backend Development**	APIs, auth, queues, caching, uploads	04-Backend-Development.md
05	**Database**	Postgres, Prisma, Drizzle, migrations	05-Database.md
06	**AI Features**	OpenAI, Claude API, RAG, agents, streaming	06-AI-Features.md
07	**Debugging**	Errors, stack traces, leaks, deploy fails	07-Debugging.md
08	**Refactoring**	Complexity, naming, duplication, clean arch	08-Refactoring.md
09	**Testing**	Jest, Vitest, Playwright, coverage, mocks	09-Testing.md
10	**Deployment**	Docker, Vercel, AWS, CI/CD, GitHub Actions	10-Deployment.md
11	**Documentation**	READMEs, API docs, guides, release notes	11-Documentation.md
12	**Productivity**	Sprint planning, reviews, PRs, task breakdown	12-Productivity.md

---

💡 Golden Rules

- **Specificity wins.** The more real context you provide, the better the output.
- **Chain prompts.** Feed the output of one into the next.
- **Constrain the model.** Every good prompt states the stack, constraints, and output format.
- **Always review.** AI code is a draft — verify before you ship.

---

*Part of the **SARION Claude Code Mastery Kit** · Section 01 of 12.*

SARION *Work Smarter. Achieve More.*

Version 1.0.0 Released July 2026

🌐 https://trysarion.com ✉ support@trysarion.com

CHAPTER 02

Debugging Workflows Library

SARION

Work Smarter. Achieve More.

🐛 Debugging Workflows Library

> **Part of the SARION Claude Code Mastery Kit** — a production-grade, no-fluff field manual for how senior engineers actually find and kill bugs.

This library is not a list of "10 tips." It is the mental model, the muscle memory, and the copy-paste tooling that separates an engineer who *guesses* from one who *knows*. Every workflow in this library follows the same spine: **reproduce → observe → isolate → hypothesize → verify → prevent.**

📑 Index — The 11 Files

#	FILE	WHAT IT COVERS	WHEN YOU REACH FOR IT
00	00-README.md	Debugging mindset, RCA, binary search, logging strategy, this flowchart	Start here. Read once, internalize forever.
01	01-Frontend-Debugging.md	React, Next.js, hydration, state, rendering, infinite renders, events, CSS, responsive, perf, memory leaks	"The UI is wrong / slow / flickering."
02	02-Backend-Debugging.md	Node, Express, FastAPI, NestJS, queues, cron, uploads, email, JWT, sessions, caching, Redis	"The server returns the wrong thing / hangs / crashes."
03	03-Database-Debugging.md	Postgres, MySQL, SQLite, Supabase, Prisma, Drizzle, indexes, slow queries, deadlocks, transactions, migrations, pooling	"The query is slow / locks / returns wrong rows."
04	04-API-Debugging.md	REST, GraphQL, webhooks, CORS, rate limits, timeouts, retries, contract drift	"The integration between two services is broken."
05	05-Authentication-Debugging.md	OAuth, JWT, sessions, cookies, SSO, RBAC, token refresh, CSRF	"Login / permissions are broken."
06	06-Performance-Debugging.md	CPU, memory, latency, flame graphs, N+1, bundle size, Core Web Vitals	"It works but it's too slow."
07	07-Production-Incidents.md	Incident command, triage, mitigation, comms, postmortems	"It's on fire *right now*."
08	08-Deployment-Debugging.md	CI/CD, Docker, env vars, secrets, rollbacks, blue/green, DNS	"It works locally but not in prod."
09	09-AI-Debugging.md	LLM prompts, hallucinations, RAG, token limits, tool calls, non-determinism, evals	"The model is doing something wrong."
10	10-Debugging-Checklist.md	The one-page pre-flight, triage, and post-mortem checklist	Print it. Pin it above your desk.

🧠 How Senior Engineers Actually Debug

The difference between a junior and a senior debugger is **not** knowledge of more tools. It is **discipline under uncertainty.** A junior sees a stack trace and starts changing lines. A senior sees a stack trace and starts asking *questions the system can answer*.

The 7 Laws of Debugging

1. **Reproduce first, or you are not debugging — you are gambling.** If you cannot make the bug happen on demand, you cannot know when you've fixed it. A "fix" for a bug you can't reproduce is a superstition.
2. **The bug is not where you think it is.** Your first hypothesis is usually wrong. Treat it as a lead, not a conclusion.
3. **Change one thing at a time.** If you change five things and the bug goes away, you have five suspects and zero convictions. You've also probably introduced two new bugs.
4. **Observe, don't assume.** "It should be X" is worthless. `console.log(X)` / `print(X)` / a breakpoint is worth everything. The machine does what it does, not what you think it does.
5. **Read the actual error.** The full message. The full stack. The line number. The junior skims; the senior reads every word, because the answer is frequently *already printed in front of them*.
6. **Binary search the problem space.** Whether it's a git history, a 2000-line file, or a request pipeline — halve the search space every step.
7. **A bug you don't understand is not fixed.** If it "went away" and you don't know *why*, it will come back — at 3 AM, on the biggest customer's account.

The Debugging Mindset: Curiosity Over Ego

Bugs are not personal insults. They are **information.** Every bug is the system telling you the truth about an assumption you got wrong. The engineer who says *"that's impossible"* is the engineer who will spend six hours stuck. The engineer who says *"interesting — the system disagrees with me, let's find out why I'm wrong"* is done in twenty minutes.

> 🔑 **The core reframe:** You are not fighting the bug. You are conducting an investigation, and the system is a cooperative witness that never lies — it only answers the exact question you ask. Ask better questions.

🔬 Root Cause Analysis (RCA)

Fixing a symptom is patching a leak with your thumb. **Root cause analysis** finds the crack in the pipe. The canonical technique is **The 5 Whys** — but done rigorously, with evidence at each step, not speculation.

Worked Example — "Users are getting logged out randomly"

WHY #	QUESTION	EVIDENCE-BACKED ANSWER
1	Why are users logged out?	Their session cookie is missing on the failing request. *(Confirmed via network tab.)*
2	Why is the cookie missing?	It's being rejected by the browser as expired. *(Confirmed: `Max-Age` header shows 0.)*
3	Why is `Max-Age` 0?	The auth service sets it from `SESSION_TTL`, which is unset in one pod. *(Confirmed: `kubectl exec … env`.)*
4	Why is it unset in one pod?	That pod was deployed from a config revision missing the env var. *(Confirmed: config diff.)*
5	Why did the config drift?	The Helm values file wasn't updated when the var was renamed; no CI check caught it. ← **ROOT CAUSE**

The **symptom fix** is "restart the pod." The **root cause fix** is "add a CI validation step that fails the build if required env vars are absent from the values file." One of these prevents the 3 AM page. Guess which.

> ⚠️ **RCA anti-pattern:** Stopping at "human error." *"Bob forgot to update the config"* is never a root cause. The root cause is *"the system allowed Bob to ship without the config."* Fix the system, not the human.

🎌 Binary-Search Debugging

The single highest-leverage technique in this entire library. When you have a large space to search — commits, lines of code, data rows, a request pipeline — **cut it in half, test, repeat.** A bug hidden in 1,000 commits is found in ~10 tests. In 1,000,000 lines, ~20 tests.

Applied to git history — `git bisect`

```bash
# The bug exists now (HEAD) but worked 200 commits ago.
git bisect start
git bisect bad                 # current commit is broken
git bisect good v2.3.0         # this old tag worked

# git checks out the midpoint. Test it, then tell git:
#   git bisect good   (if this commit works)
#   git bisect bad    (if this commit is broken)
# Repeat ~8 times for 200 commits.
```

Sample output:

```text
Bisecting: 99 revisions left to test after this (roughly 7 steps)
[a1b2c3d] refactor: extract auth middleware
 ...
a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6 is the first bad commit
commit a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6
    refactor: extract auth middleware
 3 files changed, 47 insertions(+), 12 deletions(-)
```

You now have the **exact commit** that introduced the bug, its diff, and its author. That's 90% of the investigation done automatically.

Automate it fully with a test script that exits `0` for good, non-zero for bad:

```bash
git bisect start HEAD v2.3.0
git bisect run npm test -- --testPathPattern=auth
```

Applied to code — comment-out bisection

When a 300-line function misbehaves, don't read all 300 lines. Comment out the back half, test. Bug gone? It's in the back half. Bug remains? It's in the front. Halve again. Five iterations pins it to ~10 lines.

Applied to data

`WHERE id < 500000` vs `WHERE id ≥ 500000` to find which row breaks the batch job. Same principle, different search space.

🪵 Logging Strategy

Logs are the flight recorder. Bad logging is `console.log("here")` scattered like confetti. Good logging is **structured, leveled, contextual, and correlated.**

The log-level contract

LEVEL	USE FOR	VOLUME IN PROD	EXAMPLE
`ERROR`	A failed operation a human must know about	Low	Payment capture threw
`WARN`	Recovered/degraded — suspicious but handled	Low	Fell back to cache after DB timeout
`INFO`	Business-meaningful state transitions	Medium	`order.placed`, `user.registered`
`DEBUG`	Developer detail for diagnosing	Off in prod	Query params, branch taken
`TRACE`	Firehose — every step	Never in prod	Loop iteration values

Structured logging beats string logging

JAVASCRIPT

```
// ❌ Unsearchable, uncorrelatable string soup
console.log("User " + userId + " failed to checkout: " + err.message);

// ✅ Structured — queryable in any log platform, carries context
logger.error("checkout.failed", {
  userId,
  cartId,
  requestId,          // correlation ID — ties every log in one request together
  errorCode: err.code,
  errorMsg: err.message,
  durationMs: Date.now() - start,
});
```

Now you can run `errorCode:"CARD_DECLINED" AND durationMs>3000` in Datadog/Loki/CloudWatch and *find the pattern*. String logs can't do that.

The golden rule: correlation IDs

Every request gets a unique ID at the edge, propagated through every service and every log line. When a user says "it broke at 2:47," you grep one `requestId` and see the **entire causal chain** across five microservices. Without it, you're reconstructing a crime scene from scattered receipts.

JAVASCRIPT

```javascript
// Express middleware — attach a request ID to everything downstream
app.use((req, res, next) => {
  req.id = req.headers['x-request-id'] || crypto.randomUUID();
  res.setHeader('x-request-id', req.id);
  next();
});
```

> 💡 **Production tip:** Log the *inputs* to a decision, not just the *outcome*. `logger.debug("routing", { plan, region, flag })` tells you *why* the code took a branch. Logging only "took branch B" tells you nothing when B was wrong.

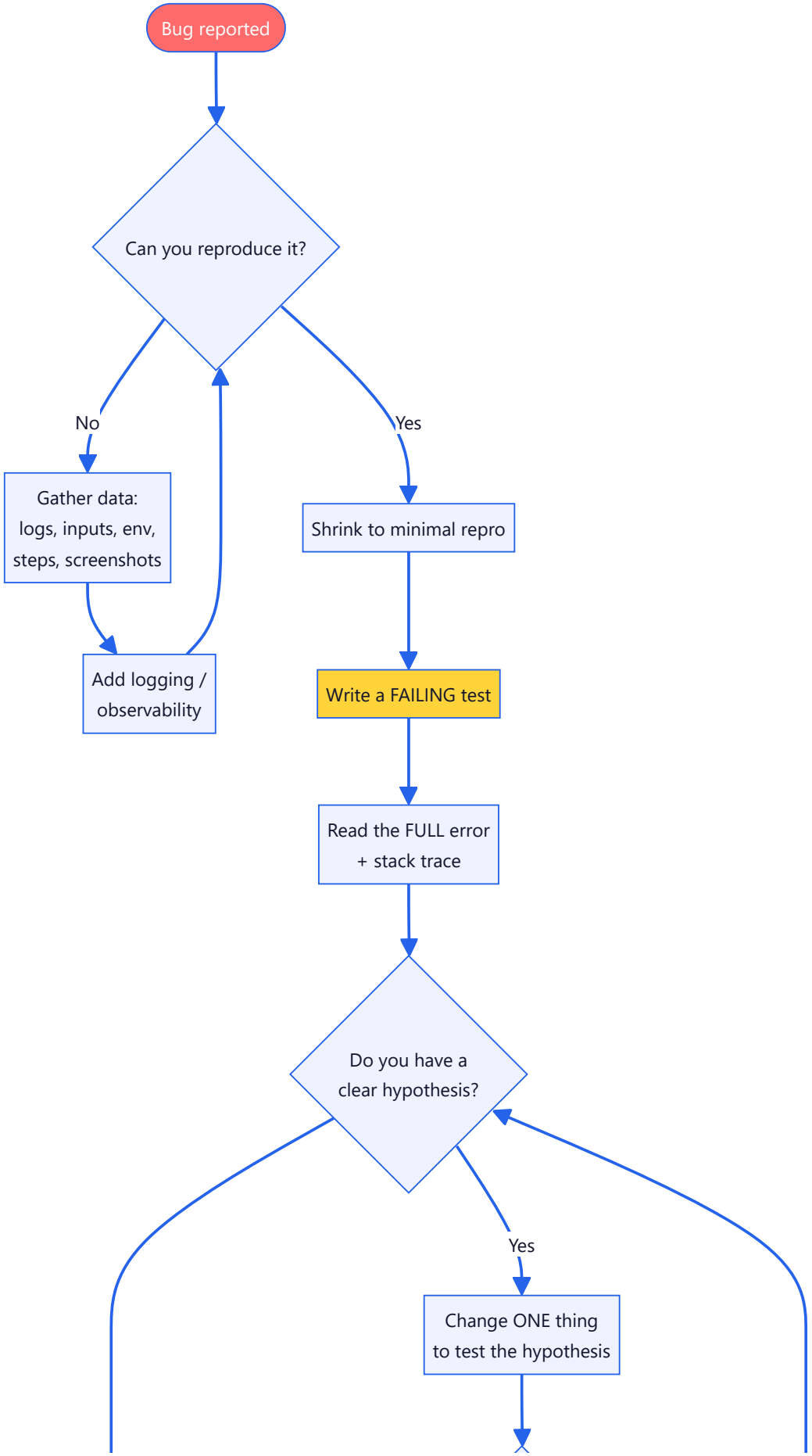
🔁 Reproduce First — The Non-Negotiable

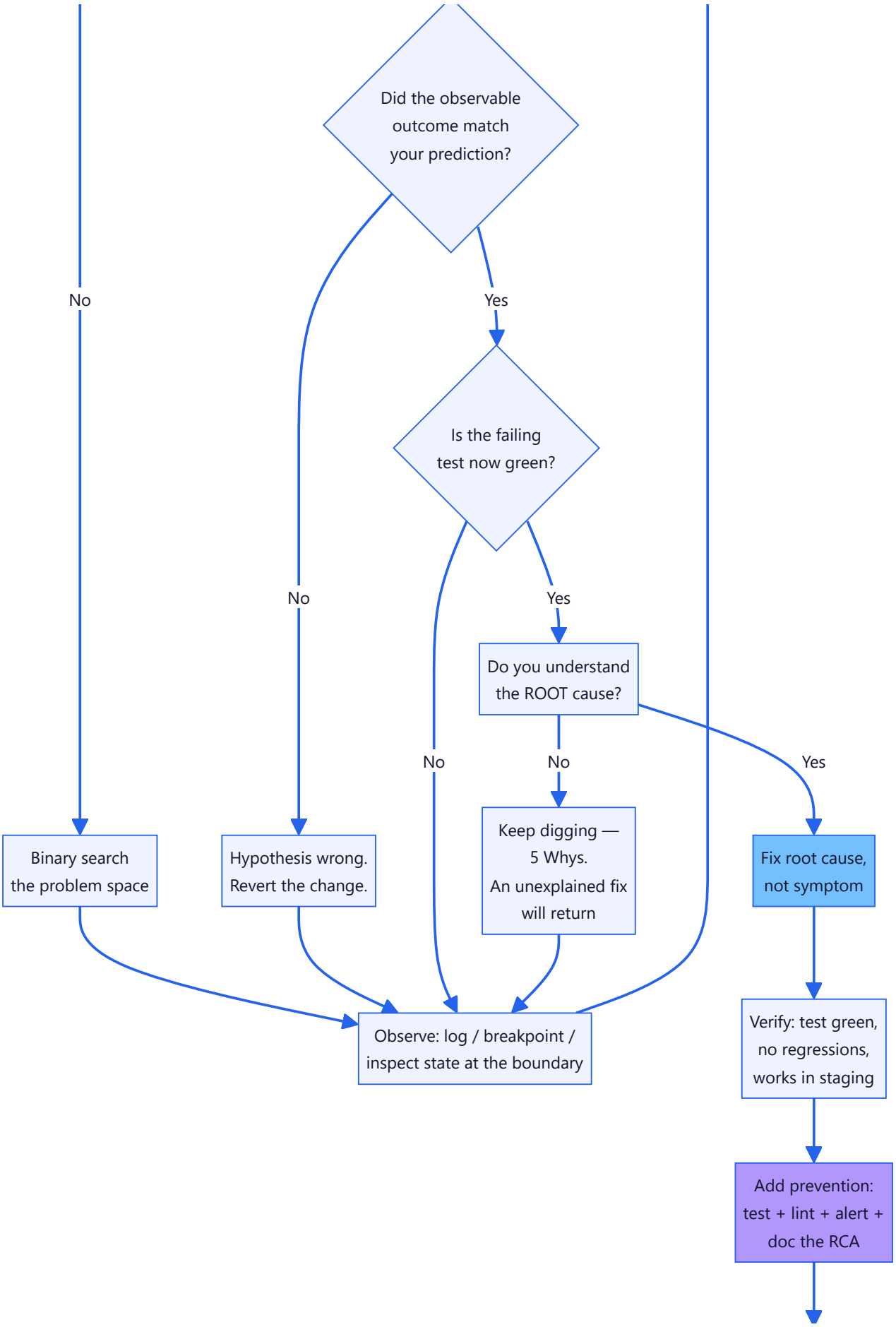
You cannot fix what you cannot see happen. Before touching code, build the **minimal reproduction:**

1. **Nail the exact inputs.** Which user, which payload, which time, which environment, which browser/version.
2. **Shrink it.** Remove everything that isn't required to trigger the bug. A 3-line repro is worth more than a 300-line one.
3. **Make it deterministic.** If it happens "sometimes," you have a *concurrency, ordering, caching, or data* problem — and that IS the bug. Chase the non-determinism.
4. **Capture it as a failing test.** The moment you can reproduce, write a test that fails. Now your fix has a definition of "done" and a permanent guard against regression.

> 🚫 **Never guess.** "Let me just try adding `await` here and see" is not debugging — it's flailing. Every change must be a **hypothesis with a predicted, observable outcome.** If you can't state "if my theory is right, then *this specific thing* will change," you don't understand the problem yet. Go observe more.

🗺️ The Master Debugging Flowchart


Bug reported
Can you reproduce it?
No
Yes
Gather data: logs, inputs, env, steps, screenshots
Add logging / observability
Shrink to minimal repro
Write a FAILING test
Read the FULL error + stack trace
Do you have a clear hypothesis?
Yes
Change ONE thing to test the hypothesis


Did the observable outcome match your prediction?
No
Yes
Is the failing test now green?
No
Yes
Do you understand the ROOT cause?
No
No
Yes
Binary search the problem space
Hypothesis wrong. Revert the change.
Keep digging — 5 Whys. An unexplained fix will return
Fix root cause, not symptom
Observe: log / breakpoint / inspect state at the boundary
Verify: test green, no regressions, works in staging
Add prevention: test + lint + alert + doc the RCA

Done. Ship it.

🤖 Using Claude Code as Your Debugging Partner

Throughout this library you'll find copy-paste **Claude Code Prompts** with `{{PLACEHOLDERS}}`. They work because they give Claude the four things it needs to be useful instead of guessy:

1. **The full error** — never paraphrase a stack trace; paste it verbatim.
2. **The relevant code** — the actual file(s), not your summary of them.
3. **What you already tried** — so Claude doesn't suggest what you've ruled out.
4. **The observed vs. expected behavior** — the delta *is* the bug.

Master debugging prompt template

```text
I have a bug in {{STACK / FRAMEWORK}}.

EXPECTED: {{what should happen}}
ACTUAL: {{what actually happens — be precise}}

REPRODUCTION STEPS:
1. {{step}}
2. {{step}}

FULL ERROR / STACK TRACE:
```

{{paste verbatim, do not trim}}

```text

RELEVANT CODE:
{{paste the actual files or @-reference them}}

WHAT I'VE ALREADY RULED OUT:
- {{tried X, no change}}
- {{confirmed Y is not the cause because Z}}

Do NOT suggest random fixes. First, give me 2-3 ranked hypotheses for
the root cause and, for each, the ONE diagnostic command or log line
that would confirm or eliminate it. I'll run them and report back.
```

> 🔑 The magic phrase is **"give me hypotheses and the diagnostic to confirm each"** — this forces the same disciplined, evidence-first loop a senior engineer runs, instead of a shotgun of speculative code changes.

✅ How to Use This Library

- **In the moment:** Jump straight to the relevant file (01–09), find your sub-topic, run the Diagnostic Commands, follow the Step-by-Step Workflow.
- **Before an incident:** Read 07-Production-Incidents.md *now*, calmly — not for the first time at 3 AM.
- **As a team standard:** Adopt the logging strategy and RCA discipline above as engineering norms. Put 10-Debugging-Checklist.md in your PR template.

Now go find the bug. It's not hiding — it's waiting to tell you exactly what you got wrong. 🔍

CHAPTER 03

The Architecture Playbook

SARION

Work Smarter. Achieve More.

🏛 The Architecture Playbook

> A Principal Architect's field manual for designing systems that survive contact with production, real users, and five years of feature creep.

Most engineers can make code *work*. Architects decide whether that code will still be maintainable, affordable, and safe when you have 100× the traffic, 5× the team, and a compliance auditor in the room. This playbook teaches you to think like the person who has to answer for those decisions.

🧠 How Senior Architects Actually Think

Junior engineers optimize for **"does it run?"**. Senior architects optimize for **"what does this cost us in 18 months?"**. The difference is a set of mental habits:

HABIT	JUNIOR INSTINCT	ARCHITECT INSTINCT
Time horizon	Ship the feature today	How does this age over 3 years?
Optimization target	Lines of code / cleverness	Total cost of ownership (TCO)
Failure model	"It works on my machine"	"What happens when this dependency dies at 3am?"
Change model	Add code	Minimize blast radius of change
Data model	Store the value	Own the source of truth + its lifecycle
Decision style	Pick the tool I know	Pick the tool the team can operate
Success metric	Feature merged	Business outcome + operability

The single most important architect skill is **naming trade-offs explicitly**. There are no "best" architectures — only architectures that are correct *for a specific set of constraints*. When someone says "microservices are better," an architect asks: *better for what? At what team size? At what traffic? At what operational maturity?*

> 💡 **Rule of thumb:** If a design has no downsides that you can articulate, you don't understand it yet.

📐 Requirements: Functional vs Non-Functional

Every system is shaped by two categories of requirements. Juniors obsess over the first. Architects are hired for the second.

Functional Requirements (FRs) — *What the system does*

- "Users can reset their password via email."
- "Admins can export invoices as PDF."
- "The system charges a card on the 1st of each month."

These map directly to features and are (relatively) easy to test: given input X, produce output Y.

Non-Functional Requirements (NFRs) — *How well the system does it*

NFRs are the **quality attributes**. They are where systems live or die.

NFR (A.K.A. "-ILITY")	THE QUESTION IT ANSWERS	EXAMPLE TARGET
Performance	How fast?	p95 API latency < 200ms
Scalability	How much load before it breaks?	10k → 1M users with no rewrite
Availability	How much uptime?	99.95% (≈ 4.4h downtime/yr)
Reliability	How often does it fail?	< 0.01% failed transactions
Security	How protected?	SOC 2, OWASP Top 10 mitigated
Maintainability	How easy to change?	New dev productive in < 1 week
Extensibility	How easy to add features?	New payment provider in < 3 days
Observability	How well can we see inside?	Any incident diagnosable from dashboards
Portability	How locked-in are we?	Cloud-agnostic core
Compliance	What law/standard applies?	GDPR, HIPAA, PCI-DSS

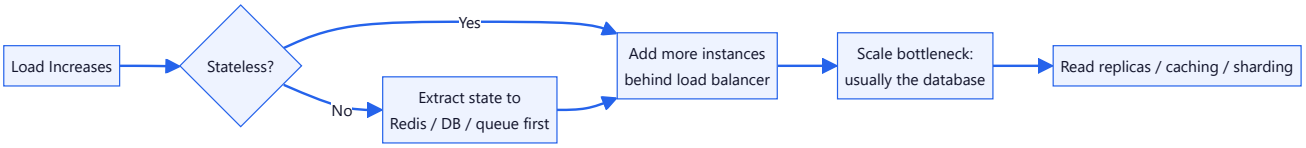
> ⚠️ **Common failure:** A team nails every functional requirement and ships a product that falls over at 500 concurrent users, leaks PII, and takes 3 weeks to add a field. All FRs met, all NFRs failed.

🎯 The Core Quality Attributes (Deep Dive)

Scalability

The ability to handle growth by **adding resources**, ideally linearly and cheaply.

- **Vertical scaling (scale up):** bigger machine. Simple, has a hard ceiling, single point of failure.
- **Horizontal scaling (scale out):** more machines. Requires statelessness, load balancing, and distributed data. Nearly unlimited but architecturally demanding.



Architect's law: *You can only scale what is stateless. State is always the hard part.*

Reliability & Availability

- **Reliability** = the probability the system performs correctly over time (no data corruption, no dropped writes).
- **Availability** = the probability it's *up* right now.

They are related but not identical. A system can be highly available (always responds) but unreliable (responds with wrong data). Key techniques: redundancy, health checks, graceful degradation, idempotency, retries with backoff, circuit breakers.

AVAILABILITY	DOWNTIME / YEAR	TYPICAL TIER
99%	3.65 days	Internal tools
99.9% (“three nines”)	8.76 hours	Standard SaaS
99.95%	4.38 hours	Paid B2B SaaS
99.99% (“four nines”)	52.6 minutes	Payments, critical infra
99.999% (“five nines”)	5.26 minutes	Telecom, exchanges

> 💡 Each extra nine roughly **10×s the cost**. Don’t buy nines you don’t need.

Security

Security is not a feature you add at the end; it is a property of the whole system. The architect’s baseline:

- **Defense in depth** — never rely on a single control.
- **Least privilege** — every service/user gets the minimum access needed.
- **Zero trust** — authenticate and authorize *every* request, even internal.
- **Secure defaults** — the easy path must be the safe path.
- **Encrypt in transit and at rest** — TLS everywhere, encrypted DB volumes.

Maintainability

The cost of understanding and safely changing the system. Driven by: clear boundaries, low coupling, high cohesion, consistent conventions, tests as a safety net, and documentation of *why* (not *what*).

Extensibility

The cost of adding *new* capability without touching existing code (Open/Closed Principle). Achieved via plugins, interfaces, event-driven hooks, and stable contracts.

⚖️ Trade-Offs: The Heart of Architecture

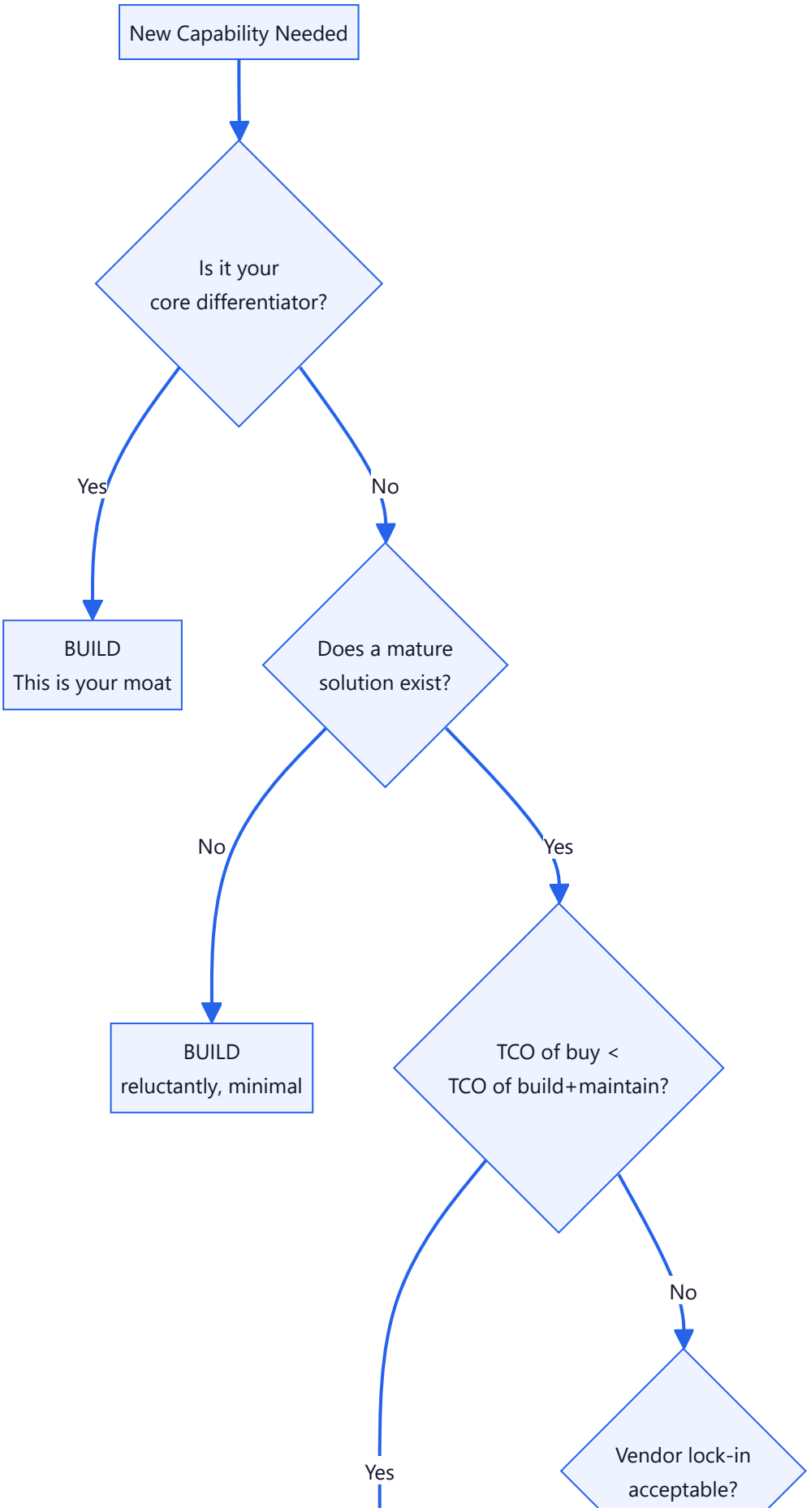
Every meaningful decision trades one quality for another. Memorize these fundamental tensions:

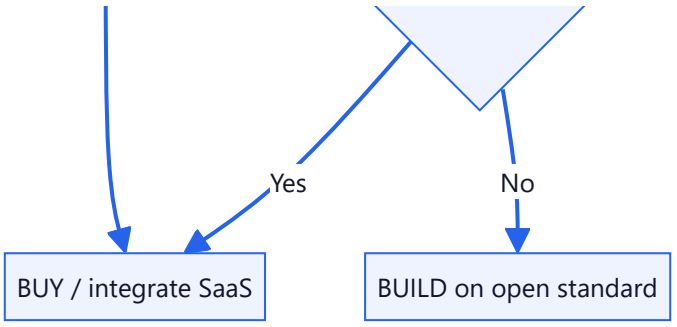
TRADE-OFF	SIDE A	SIDE B
CAP theorem	Consistency	Availability (under partition)
Consistency model	Strong (correct, slow)	Eventual (fast, temporarily wrong)
Coupling	Monolith (simple, coupled)	Microservices (flexible, complex)
Normalization	Normalized (clean writes)	Denormalized (fast reads)
Sync vs async	Sync (simple, blocking)	Async (scalable, complex)
Build vs buy	Build (control, cost)	Buy (speed, dependency)
Abstraction	DRY (reuse, coupling)	Duplication (independence, drift)
Latency vs throughput	Low latency per request	High total throughput

> 🧭 **The architect's job is not to eliminate trade-offs — it's to make them *consciously* and *documentedly*, aligned with business constraints.**

🛒 Build vs Buy

One of the highest-leverage decisions you'll make. The framework:


New Capability Needed
Is it your core differentiator?
Yes
No
BUILD
This is your moat
Does a mature solution exist?
No
Yes
BUILD
reluctantly, minimal
TCO of buy < TCO of build+maintain?
No
Yes
Vendor lock-in acceptable?



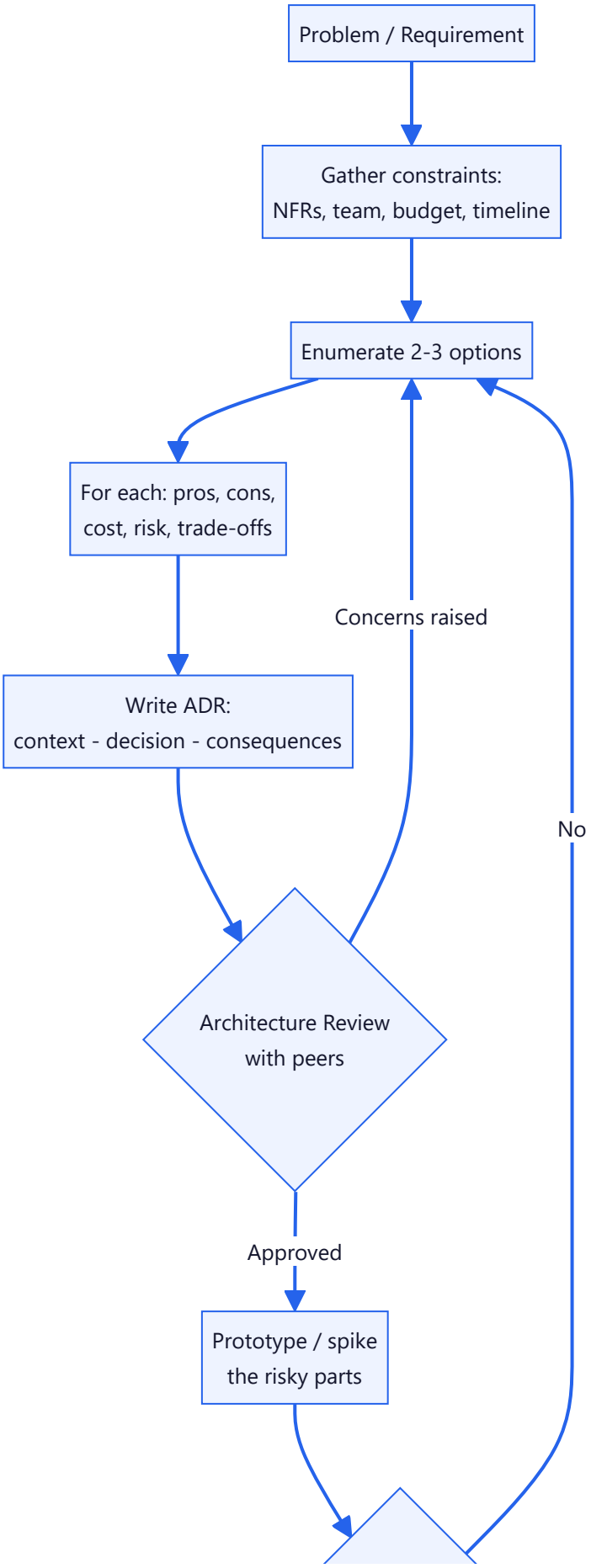
Rule: Buy your *undifferentiated heavy lifting* (auth, payments, email, search, observability). Build only what makes you *different*. A startup building its own auth system in 2026 is usually burning runway — use Auth0/Clerk/Supabase. A payments company building its own fraud engine? That's the moat — build it.

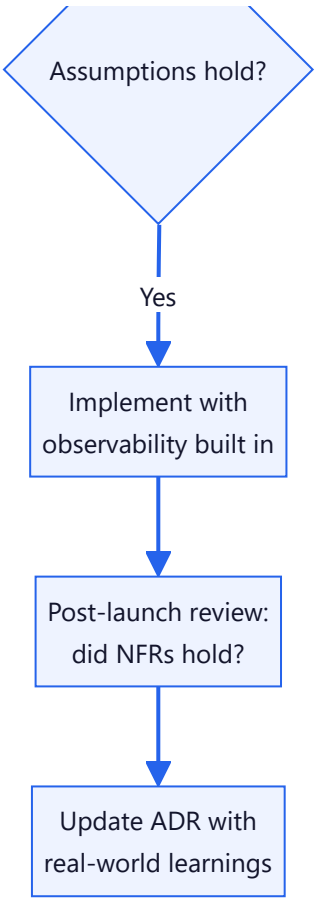
CAPABILITY	DEFAULT DECISION	WHY
Authentication	**Buy** (Clerk, Auth0, Supabase)	Security-critical, commoditized
Payments	**Buy** (Stripe)	Compliance nightmare to build
Email delivery	**Buy** (Resend, SendGrid)	Deliverability is a full-time job
Search	**Buy** (Algolia, Meilisearch)	Relevance tuning is deep
Your product's core logic	**Build**	It's why customers pay you
Observability	**Buy** (Datadog, Grafana Cloud)	Undifferentiated ops

---

🔍 The Architecture Review Process

Good architecture is a *process*, not a document. A lightweight review process (an **ADR — Architecture Decision Record** + a review meeting) catches expensive mistakes before they're code.


Problem / Requirement
Gather constraints:
NFRs, team, budget, timeline
Enumerate 2-3 options
For each: pros, cons,
cost, risk, trade-offs
Write ADR:
context - decision - consequences
Architecture Review
with peers
Concerns raised
No
Approved
Prototype / spike
the risky parts



The ADR Template (copy this)

MARKDOWN

```
# ADR-00X: [Short title of decision]
## Status: Proposed | Accepted | Superseded by ADR-0YZ
## Context
What problem are we solving? What constraints (NFRs, team, budget) apply?
## Decision
What did we decide, and what did we explicitly reject?
## Alternatives Considered
Option A (chosen) — pros/cons. Option B — why not. Option C — why not.
## Consequences
What becomes easier? What becomes harder? What do we now owe (debt)?
```

> 💡 ADRs are the single highest-ROI documentation an architect can leave behind. In 2 years, when someone asks “why the hell did we use Kafka here?”, the ADR answers it — and prevents a costly, uninformed “let’s rip it out.”

🤖 Claude Code Prompts for Architecture Work

Use these paste-ready prompts to turn Claude Code into an architecture partner.

Generate an ADR:

```text
Act as a Principal Architect. Write an Architecture Decision Record for the
following decision in my {{PROJECT_TYPE}} project:

Decision needed: {{DECISION, e.g. "how to handle background jobs"}}
Constraints: team size {{N}}, expected scale {{USERS}}, budget {{BUDGET}},
current stack {{STACK}}.

Enumerate 3 realistic options, give an honest pros/cons/cost table for each,
name the trade-offs explicitly, recommend one, and list the consequences
(what gets easier, what gets harder, what technical debt we take on).
```

Pressure-test an existing design:

```text
Here is my proposed architecture: {{PASTE DESIGN OR DIAGRAM}}.
Act as a skeptical staff engineer in an architecture review. Find the 5 most
likely ways this fails in production at {{SCALE}} scale. For each: the failure
mode, the blast radius, and the cheapest mitigation. Do not be polite.
```

Requirements extraction:

```text
From this product brief: {{BRIEF}}, extract functional requirements and
non-functional requirements separately. For each NFR, propose a concrete,
measurable target (e.g. p95 latency, availability %, RPO/RTO) appropriate for
a {{STAGE, e.g. seed-stage B2B SaaS}}.
```

📚 File Index — The Complete Playbook

#	FILE	WHAT YOU'LL LEARN
00	00-README.md	How architects think, NFRs, trade-offs, build-vs-buy, review process
01	01-System-Architecture.md	Layered, Clean, Hexagonal, MVC, MVVM, DDD, CQRS, Repository, Service layer, DI
02	02-SaaS-Architecture.md	Multi/single-tenant, agency SaaS, CRM, ERP, portals, billing, notifications, jobs
03	03-Microservices.md	Service discovery, API gateway, tracing, circuit breakers, Kafka, RabbitMQ, Redis Streams
04	04-Monolith.md	Modular monolith, when to stay monolith, the strangler-fig migration path
05	05-Serverless.md	FaaS, edge functions, cold starts, event triggers, cost model, when it wins
06	06-Database-Architecture.md	SQL vs NoSQL, sharding, replication, indexing, RLS, migrations, CQRS read models
07	07-Authentication-Architecture.md	Sessions vs JWT, OAuth2, OIDC, RBAC/ABAC, MFA, SSO, token rotation
08	08-API-Architecture.md	REST, GraphQL, gRPC, versioning, pagination, rate limiting, idempotency
09	09-Event-Driven-Architecture.md	Pub/sub, event sourcing, sagas, outbox pattern, eventual consistency
10	10-Scalability.md	Caching layers, load balancing, sharding, CDNs, autoscaling, backpressure
11	11-Folder-Structures.md	Feature-based, layer-based, monorepo, domain-driven folder layouts
12	12-Architecture-Checklists.md	Pre-launch, security, scale, and review checklists for every system

✅ Architect's Starting Checklist

- [ ] Have I written down the functional **and** non-functional requirements?
- [ ] Have I set *measurable* targets for the top 3 NFRs?
- [ ] Have I enumerated at least 2 realistic options and named their trade-offs?
- [ ] Have I decided build-vs-buy for every undifferentiated component?
- [ ] Have I written an ADR for every irreversible or expensive decision?
- [ ] Have I identified the state (the hard-to-scale part) explicitly?
- [ ] Have I planned observability *before* launch, not after the first incident?
- [ ] Do I know my target availability — and am I not over-buying nines?
- [ ] Has a skeptical peer tried to break the design in review?

> **The goal of architecture is not perfection. It's making the *reversible* decisions fast and the *irreversible* ones carefully.**

sarion
WORK SMARTER. ACHIEVE MORE.

This sample covers a fraction of what's inside. The complete SARION Claude Code Mastery Kit includes 250+ prompts, 60+ playbooks, 30+ templates, 11 companion handbooks, and 300+ pages of production-tested material.

GET THE COMPLETE SARION CLAUDE CODE MASTERY KIT

Work Smarter. Achieve More.